

	Brief Description	Big Picture	Notes/Hints	
	1 Caching at Database Driver Level			
	Many applications use a caching layer that is separate from the database connection layer, e.g. memcached vs JDBC. This is extra work for the application, but required for scalability in many cases. A much better way would be to have a local database that acts as a cache for the remote database. Your task is to create a configuration of PostgreSQL that acts as a local cache for specific relations/parts of relations. The local copy should subscribe to updates to these relations from the master copy so the local copy is up to date. The query parser should decide for each query if it can be handled from cache or needs to be sent to the master.		You can talk with Anant Utgikar (204050001@iitb.ac.in) who is a research scholar for help on this project	
	2 LSM trees in PostgreSQL			
	Built on top of B-tree infrastructure, you can use a separate B-tree for each of the LSM sub-trees and have a top-level index that wraps the lower level B-tree indices. Bloom filters are optional.	To the rest of postgres, the interface remains that of a B-Tree. The implementation of the B-Tree itself needs to be modified to that of the LSM tree, as a a hierarchy of B-Trees.	Add a layer on top of the existing B-tree structures in PostgreSQL to implement LSM trees This was done in previous years, you can take code from your seniors but MUST acknowledge it and add to it, and show how you extended it.	
	3 Query Progress Indicator			
	Queries may take a long time to finish, and users have no idea how long it will take. There has been work (dating back to 2004) on how to estimate how much time a query will take to finish. Your goal is to implement this in the PostgreSQL evaluation engine	The key to the progress indicator is to find the "driver" node in a query (you can limit yourself to queries that have only 1 pipeline stage), and see what fraction of the work has been completed by the driver node (see paper by Chaudhuri et al, linked on the right). For the purpose of this project, you can output the progress to a file although ideally it should be written to a tuple which can be polled to find the query progress.	See: Paper by Chaudhuri et al from 2004: https://dl.acm.org/citation.cfm?id=1007659 Also see https://wiki.postgresql.org/wiki/Query_progress_indication for some hints, but not all of it is relevant	
	4 Temporal Indexing			
	Create a temporal index on data with an extra valid-time dimension. Make sure to efficiently index tuples where the end time is infinity, and to support temporal queries where the time can be a range or a point. (See 7th ed of DB Concepts, Chapter 14/24for details). You can use the existing B+/R-tree implementation in PostgreSQL underneath.	Implement the temporal + other attribute index described in class/advanced indexing chapter slides online. Spatial is optional, it can be an ordinary ordered attribute such as string or numeric.		
	5 Foreign Data Wrapper: supporting semijoins			
	PostgreSQL allows a relation on site b to be defined as a foreign relation on site A, and selects on a foreign relation at A can be pushed to site B. The goal here when join relations from the two sides, to push a semi-join to B and fetch only the required tuples from B. Your first task then is to implement semijoin	Read up on semijoin optimization in distributed databases from Chapter 22 slides. Then figure out how to use existing foreign relations support in PostgreSQL to implement semijoins.	You can talk with Anant Utgikar (204050001@iitb.ac.in) who is a research scholar for help on this project	
	6 Automatic Creation of Indices in PostgreSQL			
	In this project you will modify the access layer of PostgreSQL to keep track of relation scans which could have benefited from an index, and if there are many such scans for a particular index, create the index automatically. (The index creation will be done by a separate process, not by the process doing the scan, since it can be expensive.) A commonly used technique is to check if the benefit of the index * number of queries > cost of creation of index, and when this gets satisfied you create the index. This can be extended to take cost of index maintenance into account if time permits.	Without this feature, we have seen applications perform very poorly because someone forgot to create an index needed for frequent transactions; adding the index magically improved performance. We wished that MySQL (in that application) or PostgreSQL would be smart enough to do it automatically. SQL Server on the cloud now does this.	Modify the node scan code. (1) Track number of full relation scans that have an equality predicate on a relation. (2) When the number crosses a threshold, computed as total cost of scans > cost of index creation, trigger index creation. Use fake numbers for these costs. Time permitting, extend to consider what percentage of tuples satisfy the predicate, e.g Gender='M' should not be indexed. This was done in previous years, you can take code from your seniors but MUST acknowledge it and add to it, and show how you extended it.	
	7 Query Parquet Files using DuckDB as Foreign Database Wrapper to PostgreSQL			
	DuckDB is an embedded database that lets you run queries on Files. It's become quite popular in recent years. PostgreSQL allows data in foreign databases to be accessed directly from PostgreSQL, without importing data. There is also an existing Foreign Data Wrapper (FDW) for DuckDB for directly running DuckDB queries in PostgreSQL. Your task here is (i) get the DuckDB FDW installed and working, then use DuckDB to access Parquet files, and finally write queries that access Parquet data, and show the query plans. The queries should include full select, select with a predicate, join with a local table and a Parquet table, and joins of 2 parquet tables.	1. Use this site to see how to set up DuckDB as a FDW on PostgreSQL: https://github.com/alitrack/duckdb_fdw 2. Use this site to see how to connect to Parquet files using DuckDB: https://hevo.com/learn/parquet-to-postgresql/	This project is a bit less about actually hacking PostgreSQL internals, and more about learning how to use DuckDB and link it with PostgreSQL, and how to use Parquet files. But you have to then understand what is happening under the hood, and study the performance of different queries, and how they are executed. This may be a good project for a single person, or a group of at most 2 people	
Parallel Database Projects				
	2 Distributed Caching in PostgreSQL			
	Caching of remote data can lead to significant performance improvements. In this project, you will maintain a relation on one instance A of PostgreSQL as a cached of a relation on instance B. A stores only a subset of data that is accessed locally, and when a query needs data that is not available at A, the data is fetched from B and cached. Your goal is to do caching on a per-key basis, so if data for a key is not available, it is transparently fetched from the backend, with upper layers not aware of it. You could implement this by adding support for local caching of data for a foreign table, or you can implement it on your own without using foreign tables.		Talk with Anant Utgikar if you want to do this project	